

Introduction

Docker is a Linux-based, open-source containerization platform that developers use to build, run, and package applications for deployment using containers. Unlike virtual machines, Docker containers offer: OS-level abstraction with optimum resource utilization.

Docker Terminology

Docker :An open-source project that automates the deployment of software applications inside containers by providing an additional layer of abstraction and automation of OS-level virtualization on Linux. It allows users to package an application with all of its dependencies into a standardized unit for software development.

Image : Is a blueprint for what you want to build. Ex: Ubuntu + TensorFlow with Nvidia Drivers and a running Jupyter Server.

Docker image : A Docker image is simply the software (including the filesystem and parameters) that you run within a Docker container.

Docker container : A Docker container is a mechanism for bundling a Linux application with all of its libraries, data files, and environment variables so that the execution environment is always the same, on whatever Linux system it runs and between instances on the same host. It is an instance of a Docker image. A Docker container deploys a single application or service per container.

Docker commands

1) **docker --version** : List the version of Docker for the server

2) **nvidia-smi** : NVIDIA System Management Interface (nvidia-smi) is a command line utility, based on top of the NVIDIA Management Library (NVML), intended to aid in the management and monitoring of NVIDIA GPU devices. This utility allows administrators to query GPU device state and with the appropriate privileges, permits administrators to modify GPU device state.

NVIDIA-SMI 470.103.01 Driver Version: 470.103.01 CUDA Version: 11.4									
GPU	Name	Persistence-M			Bus-Id	Disp.A	Volatile Uncorr. ECC		
Fan	Temp	Perf	Pwr:Usage/Cap			Memory-Usage	GPU-Util	Compute M.	MIG M.
0	Tesla	V100-PCIE...	Off	00000000:5E:00:0			Off	0	0
N/A	37C	P0	37W / 250W	1535MiB / 32510MiB				0%	Default N/A
Processes:									
GPU	GI	CI	PID	Type	Process name				GPU Memory
	ID	ID							Usage
0	N/A	N/A	1481	G	/usr/lib/xorg/Xorg				4MiB
0	N/A	N/A	3847847	C	...18cs/anaconda3/bin/python				1527MiB

3) docker pull : Downloads Docker images from the NGC or any other container registry to the server.

1. Open the url <https://ngc.nvidia.com/catalog/containers/>, to search the Docker registry for the image to download.
2. Select the container
3. Select the **Tags** tab and locate the container image release that you want to run.
4. In the **Pull Tag** column, click the icon to copy the docker pull command.
5. Open a command prompt and paste the pull command. The pulling of the container image begins. Ensure the pull completes successfully before proceeding to the next step.
6. Run the container image.

Eg: `docker pull nvcr.io/nvidia/pytorch:19.08-py3`

4) docker images : List all the Docker images that are currently available on the local computer. When you pull the container from a repository to a system, it is then said to be a Docker image. This means the image is local.

5) docker run : Typical command to launch the container

Eg: `docker run --gpus all -it --rm -v local_dir:container_dir nvcr.io/nvidia/tensorflow:xx.xx-tfx-py3`

Where:

- **--gpus** allows one to specify the device on which the container runs
Eg : `--gpus device= 0,2`
- **-it** means run in interactive mode
- **--rm** will delete the container when finished
- **-v** is the mounting directory
- **local_dir** is the directory or file from your host system (absolute path) that you want to access from inside your container.
- **container_dir** is the target directory when you are inside your container.
- **xx.xx** is the container version.
- **tfx** is the version of TensorFlow.

6) docker ps : List of running process image, age, status, name, and port information

7) docker ps -a : list all *Exited* and *Up* containers

8) docker exec : Connect to a running container to run commands. You can use the `bash` command to start an interactive command line terminal or bash shell.

Eg : `docker exec -it <CONTAINER_ID_OR_NAME> bash`

9) docker commit: will create a new docker with all your installations.

Eg : `docker commit <docker container id> <new docker name>:<docker tag>`

10)docker logs : You can examine logs from a past execution by issuing the `docker logs` command.

Eg : docker logs [CONTAINER ID]

11) docker rmi <Image_ID> : Remove an Image

12) docker rm <Container_ID> : Remove a Container

13) docker stop <Container_ID> : To stop a running Container

Running Jupyter on a remote Docker container using SSH

1. Port forwarding and container creation

i) ssh -L <host port>:localhost:<remote port> user@remote

ii) docker run -p <remote_port>:8888 -it --mount
type=bind,source=/home/\$user,target=/workstation <image_id>

Note the option -p , which tells the docker container to do port forwarding. This way, we can get access to apps running in the docker on a certain port from the *outside world*.

2. Move to local directory

i) cd ..

ii) cd workstation/<user_name>

3) Opening Jupyter Notebook

i) jupyter lab

ii) Open http://localhost:<host port>/ in your browser.

Important Note: Before submitting the job, ensure the availability of GPU Cards by executing nvidia-smi command. If the cards are already occupied don't disturb others' jobs. You have to wait till others jobs are completed